CO-2_Assessment Tool-2

Scenario Based Questions

CSA0712-Computer Networks

Name: C. Sravanthi

Reg No.:192525463

ANSWERS:

Scenario 1:Noisy Channel and Frame Loss

Question 1: How does a Stop-and-Wait ARQ protocol react when Frame 4 goes missing?

Answer:

- The sender sends **Frame 4**.
- Due to noise, **Frame 4 is lost** before reaching the receiver.
- The receiver does not send an ACK because it never receives the frame.
- The sender waits for an ACK.
- After the **timeout** expires, the sender retransmits **Frame 4**.
- Communication continues only after the correct ACK for Frame 4 is received.

Question 2: What happens to Frames 5 and 6 if Stop-and-Wait is used?

Answer:

- In Stop-and-Wait ARQ, the sender can transmit **only one frame at a time**.
- Since Frame 4 has not been acknowledged, **Frames 5 and 6 are not sent**.
- They remain in the sender's buffer until Frame 4 is successfully delivered and acknowledged.

Question 3: How does Go-Back-N ARQ handle the lost Frame 4 compared to Selective Repeat ARQ?

Go-Back-N ARQ

- The sender may already have sent Frames **5, 6, 7, 8...**
- The receiver expects Frame 4 first.
- It discards Frames 5, 6, and 7 because Frame 4 is missing.
- After timeout, the sender retransmits **Frame 4 and all following frames** (5, 6, 7...).

Protocol	What Happens When Frame 4 is Lost?
Stop-and-Wait	Retransmits only Frame 4 after timeout; Frames 5 and 6 are not sent until ACK is received.
Go-Back-N	Retransmits Frame 4 and all subsequent frames (5, 6, 7...).
Selective Repeat	Retransmits only Frame 4; Frames 5, 6, and 7 are buffered.

Scenario 2: MAC Sublayer and Collisions

Question 1: What network event happens when both signals meet on the wire?

Answer:

- Computers **A and B transmit simultaneously**.
- Their electrical signals interfere with each other.
- This causes a **collision**.
- The transmitted frames become corrupted and cannot be decoded correctly.

Question 2: How does CSMA/CD detect this event?

Answer:

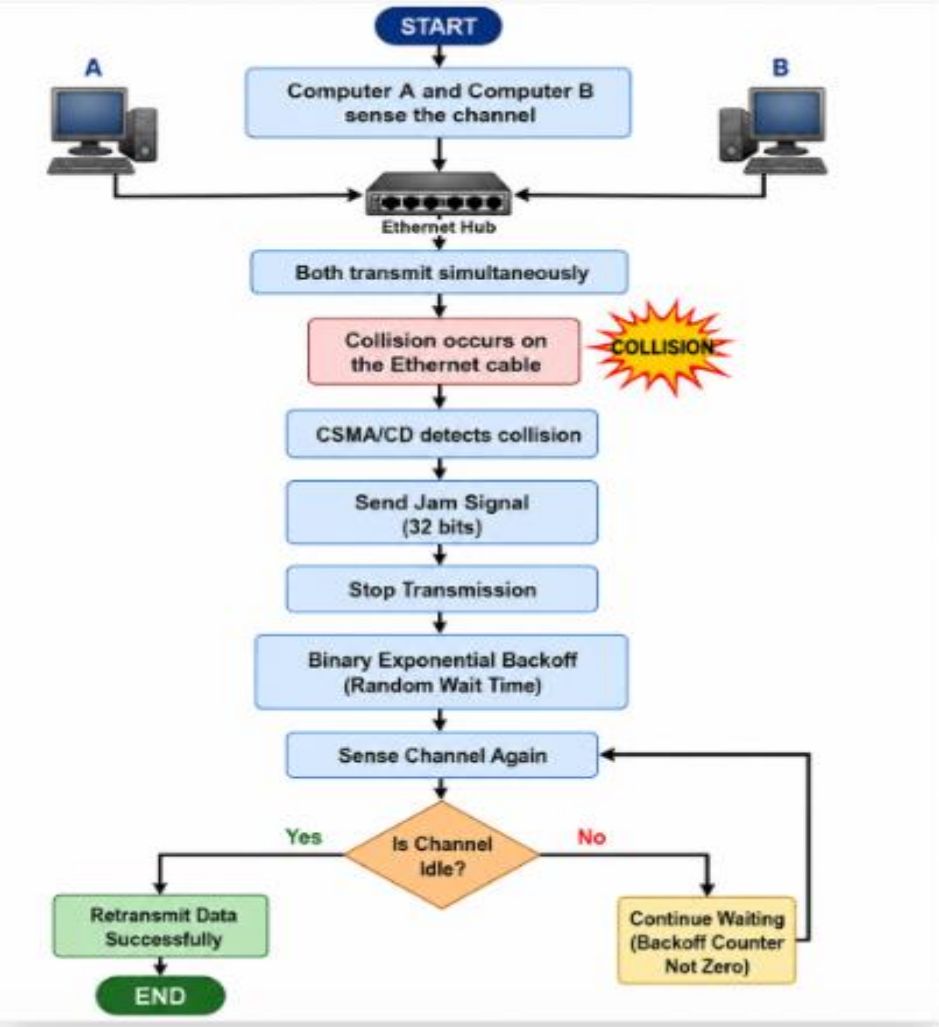
1. Each computer listens to the Ethernet cable while transmitting.
2. It compares the signal on the cable with the signal it is sending.
3. If the signals do not match, a **collision is detected**.
4. The transmitting computers immediately stop sending.
5. They send a **jam signal** so that all devices know a collision has occurred.

Question 3: Why do A and B wait for a random time period before they try to send data again?

Answer:

- If both computers retransmitted immediately, they would collide again.
- CSMA/CD uses the **Binary Exponential Backoff** algorithm.
- Each computer waits for a randomly selected time before retransmitting.
- The random delay reduces the chance of another collision.

- This improves network efficiency and fairness.



Scenario 3: Flow Control and Buffer Overflow

Question 1: What problem happens if the server sends frames faster than the printer can clear its buffer?

Answer:

- The printer receives data faster than it can process.
- Its memory buffer becomes full (buffer overflow).
- New incoming frames are dropped or lost.

- Lost frames must be retransmitted, reducing network performance.

Question 2: How does a sliding window flow control protocol stop the printer's buffer from overflowing?

Answer:

- The Sliding Window protocol limits the number of unacknowledged frames the sender can transmit.
- The printer sends acknowledgements (ACKs) indicating how much buffer space is available.
- The server sends only the allowed number of frames.
- As the printer processes data and sends ACKs, the sender's window moves forward and more frames are transmitted.
- This prevents the printer's buffer from becoming full.

Question 3: What role does the window size field in the acknowledgement frame play here?

Answer:

- The **Window Size** field tells the sender how many additional frames the receiver can accept.
- A **large window** means the receiver has enough free buffer space.
- A **small window** means the receiver is busy and the sender should slow down.
- A **window size of 0** tells the sender to stop transmitting until buffer space becomes available.

Scenario 4: Error Detection with CRC

Given

- Generator Polynomial:
  $G(x) = x^3 + x + 1$
- Binary Generator:
  1011
- Data:
  110101

Question 1: How many redundant bits (CRC bits) are appended to the data frame?

Answer:

- Degree of generator polynomial = **3**
- Therefore, **3 CRC bits** are appended.

Hence, 3 bits

Question 2: Walk through the modulo-2 division steps to find the final transmitted bit sequence.

Step 1: Append three zeros

Data = 110101
Append 000

Dividend = 110101000
Divisor = 1011

Step 2: Modulo-2 Division (XOR)

1101 ⊕1011 = 0110

Bring down next bit

Continue XOR until the end

Final Remainder = 001

Step 3: Append remainder

Original Data = 110101

CRC = 001

Final Transmitted Frame

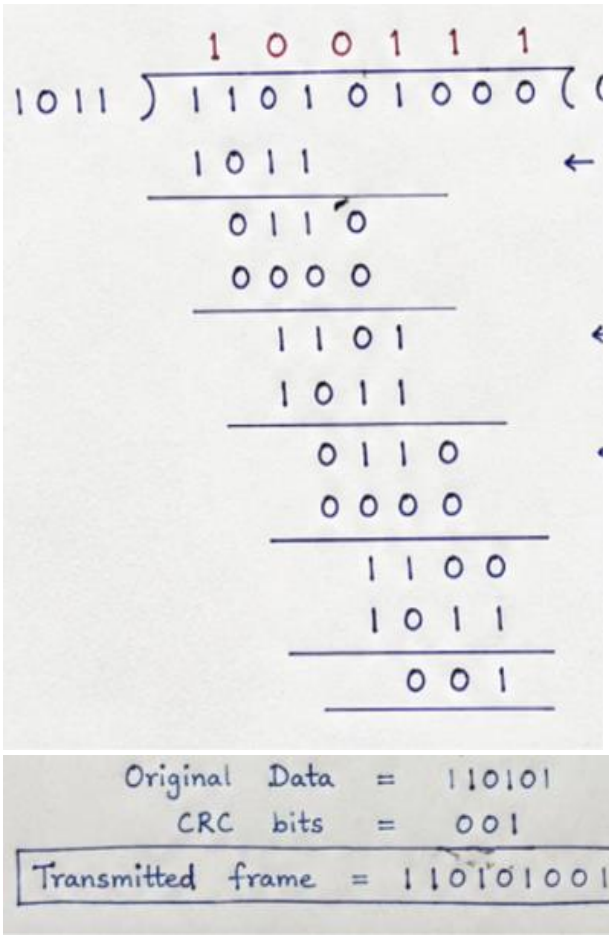
110101001

Final transmitted bit sequence = 110101001

Question 3: If a burst error flips the first two bits during transmission, how does the receiver find out that the frame is corrupted?

Answer:

- During transmission, the first two bits change.
- The receiver receives the corrupted frame.
- It performs the same CRC division using the generator **1011**.
- Because of the bit errors, the remainder is **not zero**.
- A non-zero remainder indicates the frame is corrupted.
- The receiver discards the frame and requests retransmission (if ARQ is used).

- 

Scenario 5: Framing and Byte Stuffing

Question 1: Why does the receiver think the frame ended early when it reads the data payload?

Answer:

- The byte **0x7E** is used as the **start and end flag** of a frame.
- If **0x7E** appears inside the payload, the receiver mistakes it for the end of the frame.
- As a result, the receiver terminates the frame early and interprets the remaining data incorrectly.

Question 2: How does byte stuffing (escape character insertion) solve this issue?

Answer:

- Before transmission, the sender inserts an **Escape (ESC)** byte before every special byte in the data.
- This tells the receiver that the following byte is part of the payload and not a frame delimiter.
- During reception, the receiver removes the escape byte and restores the original data.
- This allows the payload to contain special bytes without confusing the receiver.

Question 3: Show how the sender transforms the data byte 0x7E and the escape byte itself before sending.

Assume:

- **Flag Byte = 0x7E**
- **Escape Byte = 0x7D**

The sender performs byte stuffing as follows:

Original Byte	Sent Byte Sequence
0x7E	0x7D 0x5E
0x7D	0x7D 0x5D

Here:

- 0x7D 0x5E represents an escaped flag byte.
- 0x7D 0x5D represents an escaped escape byte.